

# eVault: Cryptographic Wallet & User Authentication Architecture

A Comprehensive Technical Reference Guide: From Web3 Handshake to Role-Based Security

## 1. Executive Summary: Authentication vs. Wallet Connection

In the eVault legal document infrastructure, **User Authentication** and **Ethereum Wallet Connection** are two distinct, complementary security layers that serve fundamentally different purposes:

| Dimension     | User Authentication (Credentials)                                | Ethereum Wallet Connection (Web3)                                     |
|---------------|------------------------------------------------------------------|-----------------------------------------------------------------------|
| Core Question | 'Who are you?'                                                   | 'What can you cryptographically sign?'                                |
| Mechanism     | Email + Password + Salted Bcrypt Hash + JWT Token                | MetaMask Public/Private Keypair (0x Address on Sepolia)               |
| Role          | Grants access to web interface; establishes session permissions. | Acts as the legal signing key; creates tamper-proof blockchain proof. |
| Storage       | Secure central database; ephemeral JWT session in localStorage.  | Private key never leaves user's hardware/MetaMask wallet.             |

## 2. Why Ethereum Wallet Connection is Strictly Required

A traditional username/password application fails to meet strict legal evidence standards because centralized database records can be altered, backdated, or manipulated by administrators. eVault integrates Ethereum Sepolia to provide:

- **Cryptographic Non-Repudiation (Legal Digital Signatures):** Every case filing, forensic upload, and court order is digitally signed using the user's private key. In a court of law, a petitioner, lawyer, or judge cannot deny having authored or submitted an action.
- **Tamper-Proof On-Chain Document Registration:** When legal PDFs are encrypted (AES-256-GCM) and pinned to IPFS, their cryptographic SHA-256 hash, IPFS CID, and filing timestamp are recorded immutably on Ethereum Sepolia smart contracts. Even if a local server is compromised, the ledger cannot be altered.
- **Multi-Signature Judicial Verification:** Smart contracts enforce multi-sig rules: high-stakes court decisions (e.g. bail grants, injunctions) require the verified judicial officer's private key signature directly on-chain before becoming legally effective.
- **Verifiable Chain of Custody:** Provides an unbroken, public, forensic trail of who accessed, filed, or transferred a legal record across the life of a case.

## 3. Step-by-Step Process: From Wallet Connection to User Authentication

Here is the precise technical journey executed by the system during onboarding and session initialization:

### Step 1: Web3 Handshake & Wallet Discovery

The user clicks 'Connect Wallet'. The browser issues ``eth_requestAccounts`` to MetaMask. MetaMask prompts the user to authorize the connection and returns the public address (e.g., ``0x56b7...10c33``) and Network ID (Sepolia). *No personal data or passwords are transmitted at this stage.*

### Step 2: Profile Registration & Cryptographic Identity Binding

During registration, the user selects their Legal Role (``CLIENT``, ``LAWYER``, ``POLICE``, ``JUDGE``) and provides legal credentials (Bar Council Enrollment No., Police Station Badge, or Court Bench). The system creates a 1-to-1 binding between the physical legal identity and the Ethereum address.

### Step 3: Credential Verification & JWT Session Generation

The user enters their Email and Password on the Login Gate. The Authentication Microservice validates the credentials against salted bcrypt hashes. Upon success, an encrypted JSON Web Token (JWT) is issued containing the user's Role, Bound Wallet Address, and expiration timestamp.

### Step 4: Aadhaar e-KYC & Zero-Knowledge Identity Commitment

The system initiates the identity verification gate. The Aadhaar 12-digit format is verified using the mathematical Verhoeff Checksum Algorithm followed by OTP confirmation. A zero-knowledge SHA-256 commitment ( $\text{SHA256}(\text{Aadhaar} + \text{WalletAddress})$ ) is anchored to ensure 1 human = 1 verified wallet without storing raw Aadhaar numbers.

### Step 5: Role-Based Access Control (RBAC) & Audit Recording

The frontend inspects ``currentUser.role`` and customizes permissions: - **Client**: Strictly locked to Document Upload, Personal Vault view, and Audit. - **Lawyer & Police**: Granted Document Filing, AI Classifier, and Exclusive Case Creation (+ CREATE NEW CASE). - **Judge**: Granted Bench Orders and Docket Review (Case Creation blocked). An immutable audit record (``VAULT_ACCESS_GRANTED``) is immediately logged to the ledger.

#### Key Takeaway for Legal & Technical Audits:

- **Authentication (Email/Password)** establishes *portal security* and *session state*.
- **The Ethereum Wallet** acts as the user's *cryptographic signing pen* for on-chain legal validity, tamper resistance, and zero-knowledge identity enforcement.